Listen

Learning Doc 02: Project Structure
How code is organised into folders, and why

Detailed design, Section 1

6 October 2026

Abstract

Doc 01 decided the *big pieces* of Listen (a laptop agent, a cloud API, a web app). This document zooms in one level: how those pieces become *folders and files* on disk. It explains every folder and special file in the proposed layout (`app/`, `tests/`, `pyproject.toml`, and others you will meet soon), how the folders are allowed to depend on each other, and, most importantly, a step-by-step method you could use to arrive at this structure *yourself*, without help.

Contents

1 Why folder structure matters at all **2**

2 The proposed layout **2**

3 Level 1: the repository **3**
3.1 Monorepo vs. polyrepo . . . 3
3.2 Pending decision: should `Listen/` be its own repo? . . . 3

4 Level 2: one top-level folder per program **3**

5 Level 3: inside a Python program **4**
5.1 Files, modules and packages . . . 4
5.2 The project folder vs. the package folder . . . 4
5.3 What `app/` means . . . 4
5.4 Inside `app/`: layers . . . 5
5.5 Inside `listen_agent/`: one folder per box . . . 5

6 `tests/`: code that checks code **6**

7 `pyproject.toml`: the program's ID card and shopping list **6**
7.1 What TOML is . . . 6
7.2 What `pyproject.toml` says . . . 6
7.3 Why each Python program has its *own* `pyproject.toml` . . . 7
7.4 The files that appear next to it . . . 7

8 `web/`: the JavaScript side **7**

9 `shared/`: one agreement, read by everyone **8**

10 How the folders interact **8**
10.1 Between programs: only over the network . . . 8
10.2 Inside the agent: everything goes through `core/` . . . 9
10.3 One command, traced through the files . . . 9

11 Code vs. data: what goes in the repo **9**

12 How would you come up with this yourself? **10**

13 Glossary **11**

14 Check your understanding **11**

1 Why folder structure matters at all

A computer does not care where your code lives. You could put the entire project in one 10 000-line file and Python would run it. Structure exists for *humans*:

- **Finding things.** "Where is the code that opens Google?" should have an obvious answer.
- **Changing things safely.** If the speech code is in its own folder, editing it cannot accidentally break the sync code.
- **Understanding at a glance.** A good layout is a map of the system. Someone new should be able to read the folder names and roughly understand the project.

Analogy

A library could pile every book on the floor. All the books are still "there", but you would never find one. Sections (science, history), shelves and call numbers do not change the books; they make the library *usable*. Folders are the sections of your codebase.

2 The proposed layout

```
Listen/                          <- the repository (the whole project)
|-- agent/                       <- PROGRAM 1: runs on your laptop (Python)
|   |-- listen_agent/            <- the agent's actual code (a Python package)
|   |   |-- core/                   event bus, event types, config loading
|   |   |-- voice/                  listening modes, VAD, speech engines
|   |   |-- intent/                 recipe loading + matching
|   |   |-- actions/                open_url, launch_app, ... + executor
|   |   |-- usage/                  foreground-window tracker, title cleaner
|   |   |-- storage/                SQLite access
|   |   |-- sync/                   talks to the cloud API
|   |   `-- shell/                  terminal UI now, tray app later
|   |-- tests/                   <- automated checks for the agent
|   `-- pyproject.toml           <- the agent's "ID card + shopping list"
|-- server/                      <- PROGRAM 2: runs in the cloud (Python, FastAPI)
|   |-- app/                     <- the server's actual code (a Python package)
|   |   |-- routes/                 URL handlers (the API's front desk)
|   |   |-- services/               the real work (e.g. command_builder.py)
|   |   `-- models/                 shapes of data and database tables
|   |-- tests/
|   `-- pyproject.toml
|-- web/                         <- PROGRAM 3: runs in your browser (React)
|   |-- src/
|   `-- package.json             <- JavaScript's version of pyproject.toml
|-- commands/                    <- YOUR command + routine YAML files (laptop is master)
|-- shared/
|   `-- recipe.schema.json       <- the ONE definition of a valid command
`-- docs/
    |-- learning/                <- these PDFs
    `-- specs/                   <- design documents
```

The rest of this document walks through it from the outside in.

3 Level 1: the repository

Concept: Repository (repo)

A **repository** is a project folder that **git** tracks. Git records every change you *commit*: what changed, when, and your message explaining why. You can go back to any earlier version, compare versions, and later push the whole history to GitHub so a deployment service can pull your code from there.

3.1 Monorepo vs. polyrepo

We have three programs. They can live in one repository (**monorepo**) or three (**polyrepo**).

	Monorepo (chosen)	**Polyrepo**
One change touching agent + server + web	One commit, reviewed together	Three commits in three repos, kept in step by hand
Sharing `recipe.schema.json`	Just a file in `shared/`	Copy it (copies drift) or publish it as a package
Who it suits	Solo devs, small teams, tightly linked programs	Big orgs with separate teams per program

Why it is done this way

Adding one field to a command (say, a "description") touches all three programs. In a monorepo that is one commit that tells the whole story. With three repos you would need to remember to update all three, in the right order, every time.

3.2 Pending decision: should `Listen/` be its own repo?

Right now `Listen/` sits *inside* a git repo called `LocalProject`, which also contains an unrelated Node `package.json` and `index.js`. Options:

- **Make `Listen/` its own repo (recommended).** Its history contains only Listen. When you connect a deployment service or GitHub, you connect exactly this project. Nothing in `LocalProject` is deleted.
- **Keep it inside `LocalProject`.** Works, but every Listen commit sits alongside unrelated files, and deployment tools see a repo whose root is not your project.

4 Level 2: one top-level folder per program

Concept: Deployment unit

A **deployment unit** is a piece of software that is installed and run *as one thing, in one place*. Listen has three: `agent/` runs on your laptop, `server/` runs on a cloud machine, `web/` is downloaded into a browser.

The very first cut of the layout is: **one top-level folder per deployment unit**. These three programs never share a running process; they only talk over the network. Giving each its own folder makes that boundary physical: the cloud server is deployed from `server/` alone and never even sees microphone code.

Two more top-level folders are *not* programs:

- `shared/` holds *agreements* between programs (Section 9).
- `docs/` holds documents for humans.

5 Level 3: inside a Python program

5.1 Files, modules and packages

Concept: Module and package (Python)

A **module** is one `.py` file. If you have `matcher.py`, other code can write `import matcher`. A **package** is a *folder* of modules that Python can import as a group, written with dots: `from listen_agent.intent.matcher import match`. That one line says: in package `listen_agent`, sub-package `intent`, module `matcher`, take `match`. The dots mirror the folders exactly.

You already do this in your data-analysis work: when one script imports a prepared DataFrame from another script, you are importing a module. A package is the same idea, scaled up with folders.

5.2 The project folder vs. the package folder

Why `agent/listen_agent/` and not just `agent/`? Because two different things need a home:

`agent/` (**project folder**)	Everything *about* the program: its code, its tests, its settings file, its README. Not importable.
`agent/listen_agent/` (**package folder**)	*Only* the code that actually runs. This is what `import listen_agent` loads.

Why it is done this way

Tests and configuration are not part of the running program. If they lived inside the package, they would be shipped and imported along with it. Keeping them next to the package, not inside it, keeps the running program clean. The package gets a distinctive name (`listen_agent`, not `agent`) so it can never clash with some other library also called `agent`.

5.3 What `app/` means

`server/app/` is the same idea as `agent/listen_agent/`: the server's package folder. Calling it `app` is simply the **convention** in the FastAPI world; the official tutorials and templates use it, so anyone who has used FastAPI looks for `app/` first.

Concept: Convention

A **convention** is a choice that is not technically required but that a community has agreed on, so everyone can read everyone else's projects. Following conventions is a gift to your future self and to anyone who reads your code. Inventing your own names where a convention exists is a cost, not a feature.

5.4 Inside `app/`: layers

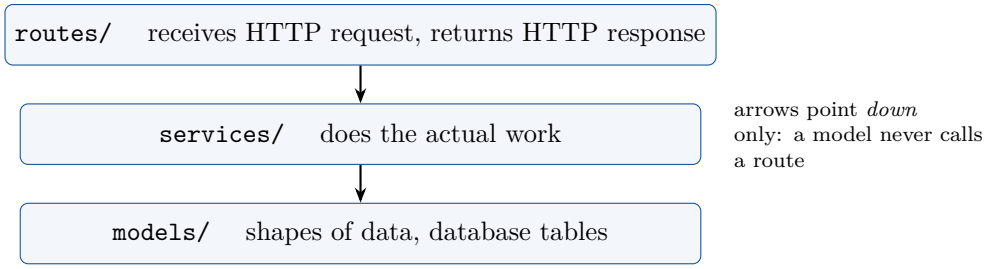
Concept: Layered architecture

Split the server's code by *kind of job*, and let each layer only call the layer below:

- **routes/**: the *front desk*. Receives an HTTP request ("`GET /analytics/apps`"), checks it is well-formed, calls a service, sends back the answer. No real logic here.
- **services/**: the *kitchen*. The actual work: computing top apps for a week, asking the LLM to draft a recipe, validating it.
- **models/**: the *pantry labels*. Definitions of data shapes and database tables.

Analogy

In a restaurant, the waiter takes your order and brings food but does not cook; the chef cooks but never talks to customers. If you change the menu layout, the chef is unaffected. If you change a recipe, the waiter is unaffected. Each layer can change without the other noticing.



5.5 Inside `listen_agent/`: one folder per box

The agent's sub-folders are *exactly* the boxes of the architecture diagram in Doc 01. That is deliberate: the diagram becomes a map of the code.

Folder	Its one job	Example files (later)
`core/`	The shared plumbing: event bus, event definitions, settings	`bus.py`, `events.py`, `config.py`
`voice/`	Microphone → text, in any of the 3 modes	`triggers/`, `vad.py`, `stt/`
`intent/`	Text → "which recipe, with which values"	`recipes.py`, `matcher.py`
`actions/`	Actually do it: open a URL, change volume	`primitives.py`, `executor.py`
`usage/`	Watch the foreground window, clean titles	`tracker.py`, `title_cleaner.py`
`storage/`	Read/write the local SQLite file	`db.py`
`sync/`	Exchange data with the cloud API	`client.py`
`shell/`	What you see: terminal now, tray icon later	`terminal.py`

These file names are a preview. Every file will be introduced, with its reason, at the moment we create it.

6 `tests/`: code that checks code

Concept: Automated test

A **test** is a small function that runs a piece of your code with a known input and checks the output is what you expect. A **test runner** (we use **pytest**) finds all of them and runs them in seconds, printing which passed and which failed.

A real example of what we will write:

```
# agent/tests/intent/test_matcher.py
from listen_agent.intent.matcher import match

def test_search_extracts_query():
    result = match("search what is an atom")
    assert result.pattern == "search {query}"
    assert result.values["query"] == "what is an atom"
```

`assert` means "this must be true, or the test fails".

Why it is done this way

Without tests, every change means manually saying "Hey Jarvis, search…" to check you did not break anything, across every command, every time. With tests, you run `pytest` and in two seconds know that all 40 behaviours still work. Tests are what let you change code *without fear*. We will also write tests *before* the code they test ("test-driven development", its own learning doc later).

Why `tests/` sits next to the package, and mirrors it. `tests/intent/test_matcher.py` tests `listen_agent/intent/matcher.py`. Mirroring means that for any code file you instantly know where its tests are. Pytest finds tests automatically by name: any file called `test_*.py`, any function called `test_*`. That naming is another convention.

7 `pyproject.toml`: the program's ID card and shopping list

7.1 What TOML is

TOML is just a text format for settings, designed to be easy for humans to read: `key = value`, grouped under `[section]` headings. Nothing more magical than that.

7.2 What `pyproject.toml` says

It is the **one file that describes a Python project** to tools. A shortened version of what the agent's will look like:

```
[project]
name = "listen-agent"                # the program's name
version = "0.1.0"                    # its version
requires-python = ">=3.11"           # which Python it needs
dependencies = [                     # libraries it needs to RUN
    "faster-whisper",
    "openwakeword",
    "sounddevice",
]

[dependency-groups]
dev = ["pytest", "ruff"]             # libraries only needed while DEVELOPING
```

```
[tool.pytest.ini_options]            # settings for the test runner
testpaths = ["tests"]
```

Concept: Dependency

A **dependency** is someone else's code that your program needs, such as `faster-whisper` for speech recognition. You do not copy it into your project; you *declare* it, and a tool downloads the right version.

Analogy

`pyproject.toml` is a recipe card's ingredient list. It does not contain the flour; it says "200 g flour". Anyone (or any deployment server) can read the list and fetch exactly the same ingredients.

You have actually seen this idea already: the `package.json` in your `LocalProject` folder is the JavaScript world's version of exactly the same file.

7.3 Why each Python program has its *own* `pyproject.toml`

Concept: Virtual environment

A **virtual environment** (venv) is a private, isolated set of installed libraries for one project. Installing something in one venv does not affect any other venv or your system Python. Each `pyproject.toml` gets its own venv.

The agent and the server need *different* ingredients:

- The **agent** needs microphone, speech and Windows libraries.
- The **server** needs FastAPI and a PostgreSQL driver.

If they shared one list, the cloud server would have to install microphone and speech libraries it can never use (wasting space, slowing deploys, and possibly failing to install on a Linux server with no audio hardware), and your lightweight agent would carry a web framework. Separate files mean each program installs only what it uses.

7.4 The files that appear next to it

`uv.lock`	Created automatically. Records the *exact* version of every library (and the libraries *they* depend on). `pyproject.toml` says "faster-whisper, roughly this version"; the lock file says "exactly 1.1.0, and exactly these 27 sub-libraries". So your laptop and the server get identical installs. Never edited by hand.
`.venv/`	The virtual environment itself: the downloaded libraries. Large, rebuilt any time from the lock file, so it is *not* saved in git.
`README.md`	A short human guide: what this program is, how to install and run it.

The tools. **uv** reads `pyproject.toml`, creates the venv, installs dependencies and writes the lock file (it replaces `pip` + `venv`). **ruff** reads your code and flags likely mistakes (unused imports, undefined names) before you even run it.

8 `web/`: the JavaScript side

React projects have their own conventions, created for us by a setup tool:

- `package.json`: the same role as `pyproject.toml`: name, dependencies, and commands like `npm run dev`.
- `src/`: the source code (pages, components). Equivalent of the package folder.
- `node_modules/`: downloaded libraries, the equivalent of `.venv/`; never committed to git.

We will cover these in detail in the web-app learning doc.

9 `shared/`: one agreement, read by everyone

Concept: Contract and single source of truth

When several programs must agree on something, write that agreement down **once**, in one place, and have every program read it. That file is the **contract**.

A command recipe is created in the web app, stored by the server, and executed by the agent. All three must agree on what a valid recipe is (it must have a `pattern`; the `action` must be one of the known primitives; . . .). `recipe.schema.json` is a **JSON Schema**: a file that describes the allowed shape of some data, which libraries in both Python and JavaScript can check data against.

Watch out

The alternative, writing the rules separately in Python (twice) and in JavaScript, works on day one. Months later someone updates one copy and forgets the others; the web app happily saves a recipe that the agent rejects. Duplicated rules *always* drift apart eventually.

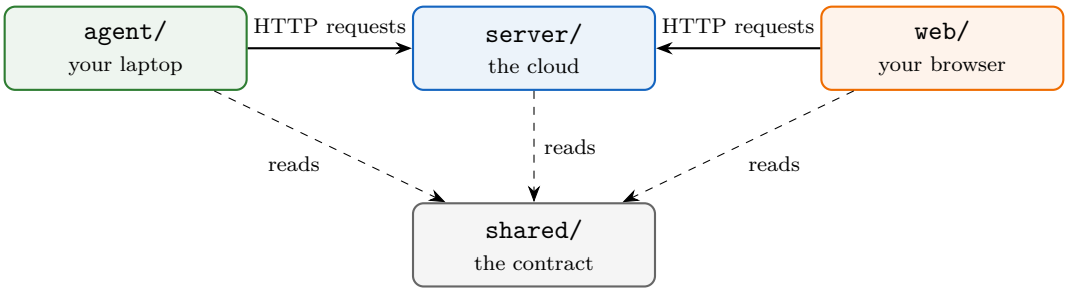
10 How the folders interact

This is the most important section. A structure is defined not just by its folders but by **who is allowed to depend on whom**.

Concept: Dependency direction

"A depends on B" means A's code uses B (imports it, or calls it over the network). If B changes, A may break. Good structures make dependencies point in *one direction* and never form a circle (A needs B needs C needs A), because a circle means nothing can be changed or tested on its own.

10.1 Between programs: only over the network

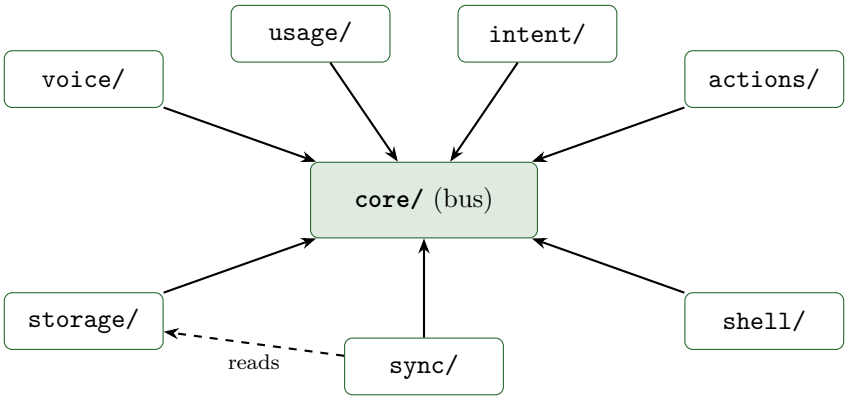


The rules:

1. `agent/` **never imports** from `server/`, and vice versa. They are different machines; the only way they talk is HTTP requests to the API.
2. `web/` talks *only* to the server's API, never to a database.

3. The server never calls the agent or the web app. They call it. (The agent *asks* the server "any command changes for me?"; the server never reaches out to your laptop. This is why sync works even when your laptop is behind home Wi-Fi.)
4. Everyone may *read* `shared/`; `shared/` depends on nothing.

10.2 Inside the agent: everything goes through `core/`



Every module depends on `core/` (to publish and subscribe to events), and `core/` depends on none of them. Modules do not import each other, with one justified exception: `sync/` reads from `storage/` to find what to upload.

Why it is done this way

Because `voice/` does not import `intent/`, you can test the matcher without a microphone, and replace the speech engine without touching the matcher. Because `core/` depends on nothing, it is stable: the piece everything relies on is the piece that changes least. That is the property you want.

10.3 One command, traced through the files

"Hey Jarvis. . . search what is an atom":

1. `shell/terminal.py` starts the program and starts each module's thread.
2. `voice/` hears the wake word, transcribes, publishes `CommandHeard` via `core/bus.py`.
3. `intent/matcher.py` (subscribed) matches the recipe `search {query}`, publishes `IntentMatched`.
4. `actions/executor.py` (subscribed) runs the primitive `open_url`, then publishes `CommandExecuted`.
5. `storage/db.py` (subscribed) writes the event to SQLite; `shell/` prints it.
6. Minutes later, `sync/client.py` reads unsent rows and sends them to `server/app/routes/`, which calls `services/`, which saves them using `models/`.
7. `web/src/` asks the server for analytics, and the command shows up on your dashboard.

11 Code vs. data: what goes in the repo

Concept: Code vs. user data vs. secrets

- **Code** (and default settings) goes in the repo, so it is versioned and shared.
- **Your data** (the SQLite database, your edited command list, logs) lives *outside* the repo, in your Windows user-data folder (`%APPDATA%\Listen`). It is personal, it changes every second, and it must never end up on GitHub.
- **Secrets** (the LLM API key, database passwords) live in a `.env` file that git is told to

ignore. The repo contains a `.env.example` with empty values, so you remember which secrets are needed.

`.gitignore` is the file that tells git which things to never track: `.venv/`, `node_modules/`, `.env`, database files. Putting a secret in git is one of the most common (and serious) beginner mistakes: once it is pushed, assume it is public, even if you delete it later.

12 How would you come up with this yourself?

Nobody invents a layout from nothing. Here is a method you can reuse on any future project. Each step is a question; the answers *produce* the structure.

1. **Where does each thing run?**
   List the places code runs: laptop, cloud, browser. *Each place becomes a top-level folder.* (This gave us `agent/`, `server/`, `web/`.)

2. **What does each program need to install?**
   If two programs need different libraries, they need separate dependency files and environments. (One `pyproject.toml` each.)

3. **What are the jobs inside each program?**
   Write the requirements as sentences, underline the verbs: *listen*, *understand*, *act*, *track*, *store*, *sync*, *display*. Each distinct job is a candidate folder.

4. **Would these two things change for different reasons?**
   This is the key test, often called the **single-responsibility principle**. Swapping the speech engine and adding a new command are unrelated changes, so speech code and matching code go in different folders. If two things always change together, keep them together.

5. **Draw the arrows: who needs whom?**
   Sketch boxes and dependency arrows. If you find a circle, introduce something in the middle that both sides depend on instead (that is exactly what `core/`'s event bus is for).

6. **What is shared between programs?**
   Anything two programs must agree on becomes a single contract file (`shared/`).

7. **What does the community already do?**
   Before inventing names, look at the official tutorial or project template for each tool (FastAPI's docs use `app/`; pytest expects `tests/` and `test_*.py`; React's setup tool creates `src/`). Reading the folder layout of a few well-known open-source projects on GitHub is one of the best ways to learn this.

8. **Start small; let it grow.**
   You do not need every folder on day one. Create a folder when its first file appears. When a file grows past a few hundred lines or starts doing two jobs, split it. Structure is refactored as you learn, not fixed forever.

Watch out

Honest note: the first time, your structure will be imperfect, and that is fine. Experienced developers mostly have better *instincts* for step 4 ("would these change for different reasons?"), built from past mistakes. Using the questions above deliberately is how you build that instinct faster.

13 Glossary

`.env / .env.example`
: File of secrets (ignored by git) / its empty template (committed).

`.gitignore`
: List of files and folders git must never track.

`app/` Conventional name for a FastAPI server's package folder.

Contract
: A single written agreement that several programs follow.

Convention
: A community-agreed naming or layout choice.

Dependency
: Someone else's code your program needs; declared, not copied.

Deployment unit
: Software installed and run as one thing in one place.

JSON Schema
: A file describing the allowed shape of JSON data.

Layered architecture
: Code split into layers (routes, services, models), each calling only the layer below.

Lock file
: Exact versions of every installed library (`uv.lock`).

Module / package
: One `.py` file / a folder of modules.

Monorepo
: One repository holding several related programs.

`pyproject.toml`
: The file describing a Python project: name, Python version, dependencies, tool settings.

Repository
: A folder whose history git tracks.

Single-responsibility principle
: Group code by its reason to change.

Test / test runner
: Code that checks code / the tool (pytest) that runs the checks.

TOML
: A simple `key = value` settings format.

Virtual environment
: An isolated set of installed libraries for one project.

14 Check your understanding

Check yourself

1. Why is there a `pyproject.toml` in `agent/` *and* in `server/`, instead of one at the top?
2. What is the difference between `agent/` and `agent/listen_agent/`?

3. A new route `GET /analytics/top-apps` needs to compute the top 5 apps. Which file in `server/app/` receives the request, and which does the computing?
4. Why may `intent/` not import `voice/`? What would you lose?
5. Where does your SQLite database file live, and why not inside the repo?
6. You need to add a tray icon. Using the method in Section 12, which folder does it go in, and why does nothing else need to change?
7. Suppose `web/` kept its own copy of the recipe rules instead of reading `shared/`. Describe a realistic bug that would appear months later.